NPOS
Non Preemptive Operating System Architektura oprogramowania

Adrian SZCZEPAŃSKI

ABSTRAKT

NPOS jest systemem czasu rzeczywistego bez wywłaszczeń. Powstał na potrzeby urządzeń wbudowanych typu 'safety-critical', gdzie zastosowanie systemu RTOS z wywłaszczeniami jest niemożliwe. System ma zaimplementowany mechanizm komunikacji międzyprocesowej (IPC), który jest podstawą wymiany danych w systemie. Na jego podstawie zdefiniowany został interfejs do korzystania z urządzeń (devices) oraz plików (filesystem).

15 October 2026

1. Scheduler

Moduł **nps** biblioteki **npos** (katalog `npos/libnpos/nps`) zawiera planistę zadań. Jest to planista kooperacyjny: zadanie wykonuje jedną porcję pracy w metodzie `process()` i oddaje sterowanie. Nie ma wywłaszczania.

1.1. Zadanie

Klasa `nps::Task` opisuje zadanie. Zadanie ma priorytet (liczba całkowita bez znaku) oraz trzy metody:

initialize()
: wywoływana raz przed pierwszym przebiegiem pętli,

isReady()
: informuje, czy zadanie ma coś do zrobienia,

process()
: wykonuje jedną porcję pracy.

Mniejsza wartość priorytetu oznacza wcześniejsze wykonanie. Porównanie zadań realizuje obiekt funkcyjny `nps::CompareTasks`.

1.2. Pętla planisty

Klasa `nps::Scheduler` nie przydziela pamięci. Lista zadań jest podawana przez użytkownika jako `Scheduler::TaskListInt&`, a gotowy kontener o stałym rozmiarze to `Scheduler::TaskList<N>`. Zadania dodaje się metodą `addTask()`, która zwraca `false` gdy lista jest pełna.

Metoda `start()` działa następująco:

- sortuje zadania rosnąco po priorytecie i wywołuje `initialize()` każdego z nich, następnie to samo dla następcy (planisty podłączonego przez `setSuccesor()`), zadania nadzorcy (watchdog) i zadania bezczynności (idle),
- w każdym obiegu pętli wykonuje zadanie watchdog, jeżeli jest gotowe,
- wykonuje pierwsze gotowe zadanie z listy; jeżeli żadne nie jest gotowe, pyta następcę,
- gdy nie ma gotowych zadań, wykonuje zadanie bezczynności.

Pętlę przerywa metoda `stop()`. Uwaga: wywołanie `stop()` w trakcie inicjalizacji jest ignorowane, ponieważ flaga pracy jest ustawiana dopiero po inicjalizacji.

1.3. Zadania nadzorcze

Zadania `setWatchdogTask()` i `setIdleTask()` nie trafiają na listę zadań. Planista uruchamia je poza kolejnością priorytetów, dlatego nie mają konfigurowalnego priorytetu.

2. Komunikacja międzyprocesowa

Moduł **ipc** biblioteki **npos** (katalog `npos/libnpos/ipc`) definiuje wiadomości, adresowanie oraz magistralę łączącą zadania.

2.1. Adresowanie

Obiekt systemu jest adresowany przez `ipc::Oid`, czyli parę (port, serwis). Port to adres zadania na magistrali, a identyfikator serwisu to indeks serwisu w zadaniu. Wartości `NULL_PORT` i `NULL_SERVICE` (255) oznaczają brak adresu. Adres `NULL_OID` oznacza rozgłaszanie.

Oid wskazuje wyłącznie na obiekty, które mogą zawieść z powodu sprzętu lub plików, na przykład otwarte urządzenie albo otwarty plik. Zadania i serwisy nie są obiektami.

2.2. Wiadomość

Struktura `ipc::Message` ma stały rozmiar. Zawiera:

sender
: adres nadawcy (Oid), do którego wraca odpowiedź,

recipient
: adres odbiorcy (Oid),

type
: typ wiadomości,

referenceCount
: licznik odwołań, gdy wiadomość jest przechowywana w puli,

input
: dane wejściowe: unia pól typowanych (otwarcie, atrybuty, odczyt lub zapis, połączenie) oraz wskaźnik i rozmiar bufora,

output
: analogiczna część wyjściowa ze statusem odpowiedzi. Domyślny status to `STATUS_MESSAGE_NOT_SUPPORTED`, więc odbiorca bez obsługi żądania nie musi go zmieniać.

Wiadomość nie ma pola określającego, czy jest żądaniem, czy odpowiedzią. O kierunku decyduje to, którą metodą trafia do odbiorcy: żądania przez `send()`, odpowiedzi przez `respond()`. Odpowiedź jest tym samym obiektem co żądanie, nie jest kopiowana.

Typy wiadomości są podzielone na zakresy: błędy systemowe, urządzenia, sterowniki, system plików, cli oraz użytkownika. Każdy moduł definiuje własne typy od początku swojego zakresu (na przykład `cli::msg::Type::SYSLOG` od `CLI_MESSAGE_TYPE`). Konkretne wiadomości dziedziczą po `ipc::Message` i dodają wyłącznie akcesory, bez nowych pól. Bufory nie są kopiowane, więc muszą istnieć do czasu obsłużenia odpowiedzi.

2.3. Serwis

Serwis (`ipc::Service`) implementuje metody `handleRequest()`, `handleResponse()`, `acceptsRequest()` oraz opcjonalnie `initialize()` i `handleUnknownRequest()`. Komunikuje się przez interfejs `Service::Api`: ,`send(sender,`message) `respond()` i `getPortId()`. Obiekt `Api` jest podawany w konstruktorze, a implementuje go zadanie serwisów. Odpowiedzi na własne żądania serwis dostaje w `handleResponse()`, a cudze żądania w `handleRequest()`.

Żądanie zaadresowane wprost do serwisu, który nie akceptuje jego typu, trafia do `handleUnknownRequest()`. Domyślnie ta metoda nic nie robi, a nadawca dostaje odpowiedź ze statusem `STATUS_MESSAGE_NOT_SUPPORTED`. Tak samo odpowiadają magistrala i zadanie, gdy adresowany port lub serwis nie istnieje albo nie akceptuje typu. Rozgłoszenie, którego nikt nie akceptuje, nie jest odrzucane. Rozgłoszenie nie powinno być żądaniem z odpowiedzią, bo jedna wiadomość służy wtedy wielu odbiorcom.

`ipc::FsmService` to serwis będący automatem skończonym. Stany dziedziczą po `FsmService::State` (metody `onEnter()`, `onExit()`, `acceptsRequest()`, `handleRequest()`, `handleResponse()` i `handleUnknownRequest()`). Obsługa zwraca identyfikator następnego stanu albo `NO_STATE_CHANGE`. Serwis akceptuje typy wiadomości wyłącznie bieżącego stanu, a po `start()` wchodzi w stan o indeksie 0.

2.4. Zadanie serwisów

Klasa `ipc::ServiceTask` jest jednocześnie zadaniem, portem na magistrali i `Service::Api`. Listę serwisów i dwie kolejki wiadomości dostaje od użytkownika jako interfejsy `ServiceListInt&`, `RequestQueueInt&` i `ResponseQueueInt&`. Kolejka odpowiedzi musi pomieścić wszystkie jednocześnie oczekujące odpowiedzi, bo odpowiedź, która się nie zmieści, jest zwracana magistrali i nadawca jej nie dostanie. Zadanie obsługuje jedną wiadomość na `process()`, odpowiedzi przed żądaniami.

- `subscribe()` nadaje serwisowi identyfikator równy indeksowi w liście,
- wiadomość do tego samego portu jest obsługiwana od razu, bez kolejkowania,
- wiadomość do innego portu trafia na magistralę,
- odbiorca z konkretnym identyfikatorem serwisu dostaje wiadomość bezpośrednio, a przy `NULL_SERVICE` wiadomość trafia do wszystkich serwisów, które akceptują jej typ.

Odpowiedzi są adresowane po polu `sender` z żądania i trafiają do `handleResponse()` serwisu nadawcy. Wiadomość, która nie mieści się w kolejce, jest zwracana magistrali (`release()`), więc nie blokuje miejsca w puli.

2.5. Magistrala

Interfejs `ipc::Bus` ma metody `send()`, `respond()` i `release()`. Implementacja `ipc::SystemBus` przyjmuje listę portów oraz pulę wiadomości od użytkownika. Numery portów są nadawane od wartości początkowej, więc dwie magistrale mogą mieć rozłączne zakresy i być połączone jako następcy.

- Wiadomość spoza puli jest kopiowana do puli przed doręczeniem, ze zerowym licznikiem odwołań. Odpowiedź jest tym samym obiektem co żądanie, więc w obrębie jednej magistrali nie jest kopiowana.
- Każde doręczenie zwiększa licznik odwołań, a `release()` zmniejsza go i zwraca wiadomość do puli, gdy licznik spadnie do zera. Na czas rozgłaszania magistrala trzyma własne odwołanie, żeby port mógł zwolnić wiadomość od razu.
- Rozgłaszanie omija port nadawcy i porty, które nie akceptują typu wiadomości.
- Żądanie do konkretnego portu jest doręczane przez `receiveRequest()`, jeśli port akceptuje typ. W przeciwnym razie magistrala woła `handleUnknownRequest()` portu i odpowiada nadawcy statusem `STATUS_MESSAGE_NOT_SUPPORTED`. Tak samo kończy się adres nieistniejącego portu.

- Odpowiedź wraca do portu zapisanego w polu `sender` przez `receiveResponse()`.
- Nieznany port jest przekazywany następcy, o ile istnieje. Następca jest sprawdzany tylko jeden raz, więc magistrale wskazujące na siebie nawzajem nie zapętlają się.
- Gdy pula jest wyczerpana, wiadomość jest odrzucana.

Moduł ipc nie zawiera logowania. Logi są zwykłą wiadomością rozgłoszeniową modułu cli, wysyłaną przez `Service::Api::send()` (patrz rozdział o terminalu).

3. Urządzenia

Moduł **dev** biblioteki **npos** (katalog `npos/libnpos/dev`) udostępnia urządzenia sprzętowe jako serwisy. Każde urządzenie ma interfejs warstwy sprzętowej (HAL) w `hal/`, wiadomości w `msg/`, serwis w `service/` oraz, dla zasobów systemowych, zadania w `task/`.

3.1. Otwieranie urządzeń

Urządzenie jest identyfikowane nazwą. Klient wysyła rozgłoszeniowe `dev::msg::Open` (funkcja `dev::api::open()`). Odpowiada serwis o pasującej nazwie, przekazując w `getOpened()` swój Oid. Dalsze żądania są kierowane już bezpośrednio do tego Oid, dzięki czemu nie obciążają magistrali rozgłoszeniami. Serwis o innej nazwie milczy.

3.2. Serwisy

Serwis	HAL	Wiadomość
LedHandler	**hal::Led**	**msg::Led**
RtcClock	**hal::Rtc**	**msg::Clock**
BackupStore	**hal::BackupRegisters**	**msg::BackupRegisters**
UartPolling	**hal::UartPolling**	**msg::Transciever**
UartInterrupt	**hal::UartInterrupt**	**msg::Transciever**
HashProcessor	**hal::Hash**	**msg::Hash**

Serwisy oparte na przerwaniach dziedziczą po `dev::InterruptService`. Przerwanie trafia do kolejki przerwań podanej przez użytkownika, a zadanie `dev::InterruptServiceTask` obsługuje je przed zwykłymi wiadomościami. Żądanie, które trwa dłużej niż jedno wywołanie, jest przechowywane przez serwis ze zwiększonym licznikiem odwołań i oddawane po przerwaniu.

Transciever (`dev::msg::Transciever`) służy do odczytu i zapisu w urządzeniach szeregowych. Odczyt prosi o *do* N bajtów. Urządzenie może zwrócić mniej i wtedy podaje faktyczną liczbę przez `setOutputLength()`.

3.3. Zadania systemowe

Timer
: przyjmuje `dev::msg::Timer` (jednorazowy lub powtarzalny) i odpowiada po upływie interwału, korzystając z `hal::SystemTick`.

Idle usypia procesor przez `hal::Power` gdy nie ma pracy.

Watchdog
: odświeża układ nadzorujący w każdym obiegu planisty.

Idle i Watchdog mają stały priorytet 0 i są uruchamiane przez planistę poza listą zadań.

4. System plików

Moduł **fs** biblioteki **npos** (katalog `npos/libnpos/fs`) definiuje interfejs systemu plików, a zadanie `fs::task::Littlefs<N>` implementuje go na bibliotece littlefs. Parametr N to liczba jednocześnie otwartych plików oraz, osobno, katalogów.

4.1. Obsługa sprzętu

Pamięć opisuje `fs::hal::Flash` (blokowy dostęp do pamięci flash) i opcjonalnie `fs::hal::ICache` (pamięć podręczna instrukcji jest wyłączana na czas programowania). Jeżeli montowanie się nie powiedzie, zadanie formatuje pamięć i montuje ją ponownie.

4.2. Wiadomości

Wiadomość	Znaczenie
file::Open	**otwarcie pliku z flagami (READ_ONLY, WRITE_ONLY, CREATE, TRUNCATE, APPEND, EXCLUSIVE)**
file::Read, file::Write	**odczyt i zapis do bufora użytkownika**
file::Close	**zamknięcie pliku**
dir::Open, dir::Close	**otwarcie i zamknięcie katalogu**
dir::Read	**odczyt kolejnego wpisu (typ, rozmiar, nazwa do 64 znaków)**
dir::Make	**utworzenie katalogu**

Otwarcie zwraca Oid pliku lub katalogu. Pole identyfikatora serwisu w tym Oid to indeks slotu w zadaniu. Odczyt katalogu kończy status `END_OF_DIRECTORY`; wpisy `.` i `..` nie są zwracane. Brak wolnego slotu skutkuje odpowiedzią `NO_MEMORY`.

Ścieżki w żądaniach otwarcia i utworzenia katalogu muszą być zakończone zerem, ponieważ trafiają bezpośrednio do littlefs. Funkcje `fs::api::file::*` i `fs::api::dir::*` wysyłają żądania w imieniu serwisu.

5. Terminal

Moduł **cli** biblioteki **npos** (katalog `npos/libnpos/cli`) zawiera terminal tekstowy oraz programy uruchamiane z jego linii poleceń.

5.1. Terminal

Zadanie `cli::task::Terminal` (wzorowane na zadaniu timera: zadanie, port i kolejka wiadomości) otwiera urządzenie wskazane nazwą i wysyła do niego `Transciever` w trybie odczytu na do 64 bajtów. Każdy odebrany bajt jest naciśnięciem klawisza. Wyjście jest kolejkowane i wysyłane porcjami, a gdy poprzedni zapis jeszcze trwa, dane czekają w buforze.

- Edycja linii: backspace, Delete, Home, End, strzałki lewo i prawo, Ctrl-A, Ctrl-E, Ctrl-U (kasuje linię), Ctrl-C (porzuca linię). Rozpoznawane są sekwencje escape CSI i SS3.
- Terminal prowadzi bufor linii i położenie kursora oraz robi lokalne echo.
- Historia pamięta pięć ostatnich poleceń, a strzałki w górę i w dół ją przełączają, przywracając linię rozpoczętą przed wejściem w historię.
- Log odebrany jako `cli::msg::SYSLOG` jest wypisywany nad edytowaną linią: terminal kasuje linię, wypisuje log w postaci `[{port:serwis} poziom] tekst` i rysuje linię ponownie wraz z kursorem. Wykrzykniki `!!!` oznaczają, że tekst został obcięty. Logi poniżej poziomu ustawionego przez `setLogLevel()` (domyślnie INFO) są pomijane.
- Terminal śledzi bieżący katalog (najwyżej 64 znaki) i pokazuje go w znaku zachęty.

Terminal jest własnym zadaniem i własnym portem na magistrali, a jednocześnie `Service::Api` swoich programów, więc dostaje wszystkie odpowiedzi do swoich żądań niezależnie od identyfikatorów serwisów w innych zadaniach. Odpowiedzi na cudze żądania są pomijane po porcie nadawcy.

Tekst logu leży w buforze nadawcy, który istnieje tylko w czasie wysyłania. Dlatego przy odbiorze terminal kopiuje go do slotu puli (`PayloadPoolInt`), podmienia `input.data` i dopiero wtedy kolejkuje wiadomość. Slot jest zwalniany po obsłużeniu logu. Gdy brakuje slotu, tekst jest za długi albo kolejka jest pełna, log jest odrzucany.

5.2. Logi

Logowanie należy do modułu cli i nie wymaga zmian w ipc. Serwis loguje przez `cli::api::syslog()` i strumień `cli::utils::LogStream`:

```
cli::api::syslog(*this, cli::log::WARNING) <<
```

Strumień składa tekst w buforze o stałym rozmiarze (MAX_MESSAGE_LENGTH = 128 znaków) i obsługuje napisy, liczby, wartości logiczne oraz bajty zapisywane szesnastkowo. Zbyt długi tekst ustawia flagę przepełnienia i dalsze dopiski są ignorowane. Dopiero `cli::log::endl` wysyła wiadomość `cli::msg::Syslog` (poziom, Oid nadawcy, flaga przepełnienia i tekst) przez `Api` serwisu. Wiadomość jest rozgłoszeniem, więc trafia do portów, które akceptują jej typ, z pominięciem portu nadawcy. Zapomniane `endl` oznacza, że wpis nie zostanie wysłany.

5.3. Programy

Program (`cli::Program`) jest serwisem. Terminal nie subskrybuje go do zadania, tylko przekazuje mu odpowiedzi działającego programu. Program implementuje ,`start(host,`args) odpowiedzi w `handleResponse()` i opcjonalnie `resume()`, wywoływane

gdy wyjście terminala zostało opróżnione. Terminal udostępnia programom interfejs `Program::Host`: `print()`, `exit()` oraz odczyt i zmianę katalogu bieżącego. Wydruk zwraca liczbę przyjętych znaków, a program kontynuuje po `resume()`, dzięki czemu wypisanie dużego pliku nie wymaga dużego bufora.

Ścieżki względne są rozwiązywane względem katalogu bieżącego terminala funkcją `text::resolve()`, która obsługuje `.` i `..`.

Program	**Działanie**
cat <plik>	wypisuje plik
echo <tekst> [> | >> <plik>]	wypisuje tekst albo zapisuje go do pliku
touch <plik>	tworzy pusty plik
ls [katalog]	wypisuje wpisy katalogu
cd [katalog]	zmienia katalog, bez argumentu przechodzi do /
pwd	wypisuje katalog bieżący
mkdir <katalog>	tworzy katalog

6. System

Biblioteka **npos-os** składa cały system z interfejsów HAL według schematu z przykładów.

6.1. Pamięć

System nie przydziela własnych kontenerów. Użytkownik podaje strukturę `os::Memory` ze referencjami do interfejsów: lista zadań, lista portów, pula wiadomości, pula serwisów urządzeń, listy serwisów i kolejki wiadomości zadań oraz kolejka timera kolejka systemu plików, kolejka terminala oraz pula slotów na teksty logów terminala. Gotowy zestaw o ustalonych rozmiarach daje szablon `os::Storage<Zadania, Serwisy, Kolejka, Wiadomości, SerwisyUrządzeń, Programy, Logi>`, którego metoda `memory()` zwraca `Memory`. Sloty puli logów mają rozmiar `cli::utils::LogStream::MAX_MESSAGE_LENGTH`, a terminal (jeżeli włączono cli) udostępnia metoda `getTerminal()`. Pamięć musi przeżyć system.

Serwisy urządzeń są tworzone w puli o slotach wielkości największego z nich (`os::ServiceSlot::SIZE`). Serwisy, które nie zmieszczą się w puli, są pomijane.

6.2. Sprzęt

Struktura `os::Hal` jest tworzona z obowiązkowym `SystemTick&` i opisuje pozostałe urządzenia:

flash,icache
: dodają zadanie systemu plików,

power
: dodaje zadanie idle,

watchdog
: dodaje zadanie watchdog,

leds (`LedBank`),

rtcs,backups,pollingUarts
: (`Device<T>` z nazwą),

uarts,hashes
: (`InterruptDevice<T>` z nazwą i kolejką przerwań).

Urządzeń każdego rodzaju może być wiele, każde pod własną nazwą.

6.3. Konfiguracja i uruchomienie

Struktura `os::Config` zawiera flagę `cli`, nazwę urządzenia terminala oraz priorytety zadań: użytkownik 1, terminal 2, system plików 4, urządzenia 5, urządzenia z przerwaniami 6, timer 9. Wszystkie mają wartości domyślne.

Serwisy użytkownika powstają przed systemem, więc potrzebują obiektu `Api` już w konstruktorze. Rozwiązuje to `os::UserApi`, który system wiąże z zadaniem użytkownika.

Tworzenie systemu wygląda tak:

```
os::Storage<> storage;
os::UserApi api;
MyService service(api);
ipc::Service* services[] = { &service };

os::Hal hal(tick);
hal.rtcs = rtcs;

os::System system(hal, storage.memory(), services, api);
system.start();
```

System tworzy zadania: użytkownika, urządzeń bez przerwań, urządzeń z przerwaniami, systemu plików (gdy podano pamięć flash), timera oraz idle i watchdog (gdy podano odpowiednie HAL). Zadania bez serwisów nie są dodawane do magistrali. Klasa `os::System` to alias `os::BasicSystem<8>`; parametr szablonu to liczba otwartych plików.

7. Emulator

Biblioteka **npos-emulator** zawiera implementacje HAL do uruchamiania systemu na komputerze: Flash, ICache, Led, Rtc, SystemTick, Hash, UartInterrupt, UartPolling, BackupRegisters, Power, Watchdog oraz ConsoleUart (stdin i stdout).

Mocki, które coś wypisują, mają argument konstruktora `verbose` (domyślnie wyłączony). Wydruki są włączane tylko w programach przykładowych, a testy jednostkowe są ciche.

ConsoleUart zwraca dane tak szybko, jak są dostępne, a resztę bufora wypełnia zerami, co terminal ignoruje.

8. Organizacja projektu, testy i przykłady

8.1. Biblioteki

Projekt składa się z trzech bibliotek budowanych z głównego `Makefile`:

npos
: rdzeń systemu w katalogu `npos/libnpos`, podzielony na moduły `nps`, `ipc`, `dev`, `fs` i `cli`; dołącza się je jako `<libnpos/moduł/Plik.hpp>`,

npos-emulator
: atrapy HAL do uruchamiania na komputerze,

npos-os
: klasa `os::System` składająca system.

Biblioteki są nagłówkowe, a zależności zewnętrzne to ETL (nagłówki) i littlefs. Moduły nie przydzielają pamięci. Kontenery, kolejki i pule podaje użytkownik.

8.2. Testy jednostkowe

Testy są napisane w CppUTest. Testy rdzenia są w `npos/tests`, w podkatalogach odpowiadających modułom (`nps`, `ipc`, `dev`, `fs`, `cli`), i tworzą jeden program testowy `test-npos`. Biblioteki `npos-emulator` i `npos-os` mają własne katalogi `tests`. Testy uruchamia Makefile z głównego katalogu przy budowaniu biblioteki, a niepowodzenie testu przerywa budowanie.

- Plik testowy modułu ma nazwę `Moduł.test.cpp` i leży w katalogu odpowiadającym modułowi.
- Pliki obiektowe są nazywane bez katalogu, a w rdzeniu jest teraz jeden program testowy, więc nazwa testu musi być unikalna w całej bibliotece. Gdy dwa moduły mają tę samą nazwę, do nazwy testu dodaje się przyrostek (na przykład `ApiDev.test.cpp`, `ApiFs.test.cpp`, `OpenDev.test.cpp`, `DirOpen.test.cpp`, `TimerTask.test.cpp`).
- Katalogi z testami są wymienione w `npos/tests/Makefile` jako kolejne wzorce `wildcard`. Nowy podkatalog z testami trzeba tam dopisać, w przeciwnym razie jego testy nie zostaną skompilowane.
- Wspólne atrapy (`RecordingApi`, `RecordingBus`, `FakeBus`, `RamFlash`, `Console` i inne) są w `npos/tests/mock`.
- Nagłówki projektu należy dołączać przed `CppUTest/TestHarness.h`, ponieważ ten nagłówek podmienia operator `new`.

8.3. Dokumentacja

Dokumentację buduje się poleceniem `make -C docs`. Zródłami są pliki rozdziałów `docs/source/NN_nazwa.roff` łączone w `main.roff`, a wynikiem jest `docs/manual.pdf`.

8.4. Przykłady

Katalog `examples` zawiera programy korzystające z `os::System` i emulatora: blinking-led, get-time, compute-hash, uart-read-write, filesystem, dual-system oraz cli (terminal na stdin i stdout, na przykład `printf 'ls\n' | ./bin/Pc32/debug/cli.elf`). Wskazówka: polecenia podawane przez potok należy rozdzielać pauzami, ponieważ terminal ignoruje bajty wpisane w trakcie działania programu.

Spis treści

1. Scheduler 2
1.1. Zadanie 2
1.2. Pętla planisty 2
1.3. Zadania nadzorcze 2
2. Komunikacja międzyprocesowa 4
2.1. Adresowanie 4
2.2. Wiadomość 4
2.3. Serwis 4
2.4. Zadanie serwisów 5
2.5. Magistrala 5
3. Urządzenia 6
3.1. Otwieranie urządzeń 6
3.2. Serwisy 6
3.3. Zadania systemowe 6
4. System plików 8
4.1. Obsługa sprzętu 8
4.2. Wiadomości 8
5. Terminal 10
5.1. Terminal 10
5.2. Logi 10
5.3. Programy 10
6. System 12
6.1. Pamięć 12
6.2. Sprzęt 12
6.3. Konfiguracja i uruchomienie 12
7. Emulator 14
8. Organizacja projektu, testy i przykłady 16
8.1. Biblioteki 16
8.2. Testy jednostkowe 16
8.3. Dokumentacja 16
8.4. Przykłady 16